Prof. Dr. Stefan Leutenegger
Teaching Assistants: Joshua Näf, Micha Bosshart, Matteo Iovino

Exercise 12: Robot Learning

Session date: 26/05/2026

Task 1: Solving MDPs

Figure 1: The frozen lake gym environment.

In class, we learned that we can perform Value Iteration by solving the following equation until convergence

$$V_{t+1}(x) \leftarrow \max_u \sum_{x_+} p(x_+|x,u)[R(x,u,x_+) + \gamma V_t(x_+)], \tag{1}$$

which we can equivalently formulate in terms of the Q-function as

$$Q_t(x,u) = \sum_{x_+} p(x_+|x,u)[R(x,u,x_+) + \gamma V_t(x_+)], \tag{2}$$

$$V_{t+1}(x) \leftarrow \max_u Q_t(x,u). \tag{3}$$

Also recall that Policy Iteration solves MDPs by first initializing a policy $\pi_0(x)$ and then iterating the following steps. First, we perform policy evaluation, where we solve the following equation for all x until convergence by iterating l.

$$V_{l+1}^{\pi_t}(x) = \sum_{x_+} p(x_+|x,\pi_t(x))[R(x,\pi_t(x),x_+) + \gamma V_l^{\pi_t}(x_+)] \tag{4}$$

Then, we extract the new optimal policy by performing

$$\pi_{t+1}(x) = \arg\max_u \sum_{x_+} p(x_+|x,u)[R(x,u,x_+) + \gamma V_\infty^{\pi_t}(x_+)] \tag{5}$$

over all x. We repeat these two steps until the policy converges.

Define the transition probability tensor $P \in \mathbb{R}^{n_u \times n_x \times n_x}$, where the entry P_{ijk} represents the conditional probability $p(x_+ = k \mid x_t = j, u_t = i)$ of transitioning from state j to state k upon the application of action i.

Additionally, we similarly define the reward tensor $\mathcal{R} \in \mathbb{R}^{n_u \times n_x \times n_x}$ with $\mathcal{R}_{ijk} = R(x = j, u = i, x_+ = k)$ and $\mathcal{V}_t \in \mathbb{R}^{n_x}$ with $\mathcal{V}_{t,k} = V_t(x = k)$.

a) Write down the Value Iteration update in (2) and (3) in matrix form in terms of these quantities.

Hint: You might find the following operations using Einstein summation handy: $A_{ij} = B_{ijk}C_k$ for tensor-vector multiplication and $A_{ij} = B_{ijk}C_{ijk}$ for tensor-tensor multiplication.

b) Write down the Policy Iteration formulas in matrix form.

Hint 1: Iterating policy evaluation until convergence reaches a fixed point. What condition is satisfied at this fixed point? How can you express this as a tensor equation?

Hint 2: The policy conditioned transition and reward tensors π_t and $\mathcal{R}^{\pi_t}$ degenerate to $\mathbb{R}^{n_x \times n_x}$. You will need their product $A_j = P^{\pi_t}_{jk}\mathcal{R}^{\pi_t}_{jk}$. What quantity does this represent?

c) Implement the Value and Policy Iteration updates you derived in the previous subtasks in the Frozen Lake environment in `e12_task_1_mdp.py`.

Task 2: Q-Learning

We have seen in the previous exercise how we can find optimal policies in MDPs with finite state and action spaces under known probabilities. However, for most problems of interest we do not know the transition probabilities (i.e. the model) a priori. Fortunately, we do not need the transition dynamics in Q-Learning which performs the following update after observing a transition x, u, x_+,

$$\tilde{Q}_i(x, u) = \tilde{R}(x, u, x_+) + \gamma \max_{u_+} Q_i(x_+, u_+), \tag{6}$$

$$Q_{i+1} = Q_i(x, u) + \alpha \underbrace{\Big(\tilde{Q}_i(x, u) - Q_i(x, u)\Big)}_{\text{Temporal difference}}, \tag{7}$$

where we use the temporal difference formulation of the learning update.

Fill in the missing pieces in `e12_task_2_q_learning.py` where the agent is tasked with learning the rules of blackjack. How does the performance of the policy compare to a human? How to a random policy?

(Optional) Task 3: Deep Q-Learning (DQN)

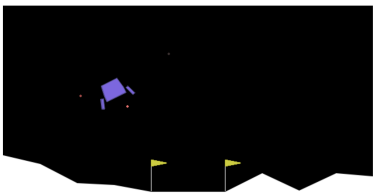

Figure 2: The gymnasium lunar lander environment.

You have seen in task 2 how we can learn optimal policies for MDPs with unknown transition dynamics in the tabular setting. However, almost no system of engineering interest has a tabular state and action space. This means we either have to discretize the state/action spaces or find different methods that are suited to continuous state and action spaces.

In this task, we investigate an extension of Q-Learning to continuous state spaces. Instead of discretizing the state space, we approximate the Q-function using an multi layer perceptron (MLP). Specifically, let $\texttt{QNet}_\theta : \mathbb{R}^{n_x} \to \mathbb{R}^{n_u}$ be a Q-network parameterized using parameters θ. Instead of extracting the Q-value from a matrix with $Q(x = j, u = i) = Q_{ij}$, we now get the Q-value using $Q(x = j, u = i) = \texttt{QNet}_\theta(j)[i]$. We perform network updates by minimizing the following error using gradient descent

$$\ell(\theta) = 0.5 \left(\tilde{Q}_i(x, u) - Q_i(x, u) \right)^2 \tag{8}$$

which results in the following update formula

$$\theta_{i+1} = \theta_i + \alpha \left(\tilde{Q}_i(x, u) - Q_i(x, u) \right) \nabla_\theta Q_i(x, u) \tag{9}$$

a) What does θ and $\nabla_\theta Q_i(x, u)$ correspond to in the tabular case?

b) Fill in the missing pieces in `e12_task_3_dqn.py` where you are tasked with landing a lunar lander on the surface of the moon. You have a continuous state space but are limited to a discrete action space (max firing the individual thrusters or doing nothing). How does the choice of the individual parameters in `DQNConfig` influence the performance of the policy?

c) DQN uses two networks to estimate the Q-function. What problems does this try to alleviate?

(Optional) Task 4: Deep RL (SAC vs PPO)

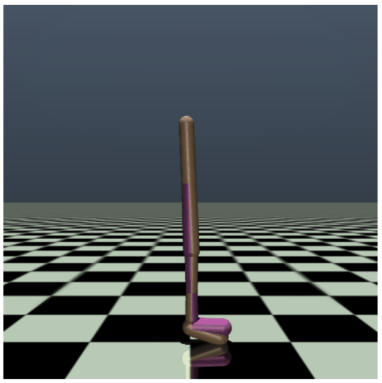

Figure 3: Walker 2D environment.

Now we want to tackle problems with continuous state and action spaces. We refer to [1, 2] for detailed derivations of PPO and SAC. Here, we will primarily focus on trying to tune these methods, so the following exposition is solely to give you some intuition into these methods. To guide your tuning process, recall the core optimization objectives for both algorithms:

1. Proximal Policy Optimization (PPO)

PPO is an on-policy actor-critic algorithm designed to ensure stable updates by preventing the new policy π_θ from deviating too far from the old policy $\pi_{\theta_{\text{old}}}$. It achieves this by defining a probability ratio:

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)} \tag{10}$$

Instead of a standard policy gradient, PPO maximizes the **Clipped Surrogate Objective Function**:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta)\hat{A}_t, , \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right] \tag{11}$$

where $\hat{A}_t$ is the estimated generalized advantage at timestep t, and ϵ is the clipping hyperparameter (typically set between 0.1 and 0.2). The $\min$ operator establishes a pessimistic bound, ensuring that policy updates are penalized if the ratio $r_t(\theta)$ moves outside the interval $[1-\epsilon, 1+\epsilon]$.

2. Soft Actor-Critic (SAC)

SAC is an off-policy actor-critic framework based on **Maximum Entropy Reinforcement Learning**. Rather than simply maximizing expected cumulative rewards, the objective function is augmented to maximize both the expected reward and the expected entropy (randomness) of the policy:

$$J(\pi) = \sum_{t=0}^{T} \mathbb{E}(s_t, a_t) \sim \rho\pi \left[R(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot \mid s_t)) \right] \tag{12}$$

where $\mathcal{H}(\pi(\cdot \mid s_t)) = -\log \pi(a_t \mid s_t)$ represents the policy's entropy, and α is the **temperature parameter** that governs the trade-off between exploration (high entropy) and exploitation (high reward). To update its value functions while avoiding overestimation bias, SAC fits two Critic networks ($Q_{\theta_1}, Q_{\theta_2}$) to a target value derived from a modified Bellman optimality operator:

$$y = R(s,a) + \gamma \left(\min_{j=1,2} Q_{\bar{\theta}j}(s', a') - \alpha \log \pi\phi(a' \mid s') \right) \tag{13}$$

where $\bar{\theta}$ denotes the exponentially moving target network weights, and ϕ denotes the parameters of the Actor network.

We wish to control the Walker 2D in this task. The policy can observe the continuous positions and velocities of each joint and apply a continuous torque to them. Note that finding a policy that actually walks is nontrivial but most policies get stuck in a local minimum where the walker starts hopping.

a) What happens if you try to run DQN in `e12_task_4_rl.py`? Why does it not work?

b) Try out some different parameter configurations of PPO and SAC in `e12_task_4_rl.py` and then try to answer the following questions.

i) **On-Policy vs. Off-Policy Efficiency**
Explain the primary difference between PPO and SAC in terms of data usage. Why is SAC generally considered more sample-efficient than PPO?

ii) **Hyperparameter Sensitivity and Ease of Use**
Contrast SAC and PPO regarding their sensitivity to hyperparameters. Which algorithm is generally considered the "baseline" for a new problem, and why?

iii) **The Clipped Objective in PPO**
PPO introduces a "clipped" surrogate objective function. What is the specific purpose of this clipping mechanism, and how does it contribute to the stability of the training process?

iv) **Maximum Entropy Reinforcement Learning**
SAC is based on the Maximum Entropy RL framework. What is the augmented reward objective used in SAC, and how does the temperature parameter α influence the agent's behavior?

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[2] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, and P. Abbeel, "Soft actor-critic algorithms and applications," *arXiv preprint arXiv:1812.05905*, 2018.